
title: "Capitolo 2 - Quando Docker non basta più" subtitle: "Dai container all'orchestrazione con Kubernetes" author: "OpenShift, Kubernetes e GitOps in pratica" chapter: 2 version: 1.0

Capitolo 2

Quando Docker non basta più

"Ogni tecnologia nasce per risolvere un problema. Kubernetes non fa eccezione."

Obiettivi del capitolo

Al termine di questo capitolo sarai in grado di:

- comprendere i limiti di Docker Compose;
 - capire perché nasce Kubernetes;
 - installare un cluster Kubernetes locale;
 - creare e gestire Pod;
 - utilizzare Deployment e Service;
 - organizzare le risorse mediante Namespace;
 - comprendere il funzionamento interno del cluster;
 - distribuire l'applicazione BookStore su Kubernetes;
 - osservare il modello dichiarativo in azione.
-

Prerequisiti

Per seguire tutti i laboratori è necessario avere installato:

- Git
- Docker Desktop oppure Docker Engine
- kubectl
- Minikube oppure Kind
- Visual Studio Code

Tutti gli esempi presenti in questo libro sono stati progettati per essere eseguiti interamente su **localhost**.

Non utilizzeremo servizi cloud.

L'obiettivo è comprendere Kubernetes senza sostenere alcun costo.

2.1 Quando Docker non basta più

Nel capitolo precedente abbiamo containerizzato l'applicazione **BookStore**.

L'architettura era composta da tre componenti principali:

- frontend React;
- backend Spring Boot;
- database PostgreSQL.

Grazie a Docker Compose è stato sufficiente un solo comando per avviare l'intera applicazione.

```
docker compose up -d
```

Per un progetto locale questa soluzione è eccellente.

Docker Compose permette infatti di:

- creare automaticamente una rete privata;
- avviare più container contemporaneamente;
- gestire variabili d'ambiente;
- montare volumi persistenti;
- descrivere l'intera infrastruttura in un unico file YAML.

Durante lo sviluppo quotidiano rappresenta uno strumento estremamente efficace.

Ma cosa succede quando l'applicazione inizia a crescere?

Il primo problema

Immaginiamo che BookStore venga pubblicato e inizi ad essere utilizzato da migliaia di utenti.

Il backend riceve sempre più richieste.

La CPU raggiunge rapidamente il 100%.

Il tempo di risposta aumenta.

L'applicazione diventa lenta.

Con Docker Compose possiamo certamente aumentare le risorse del container.

Ma questa soluzione funziona soltanto fino a un certo punto.

Prima o poi sarà necessario distribuire il carico su più istanze del backend.

Docker Compose non è stato progettato per questo scenario.

Il secondo problema

Immaginiamo ora che il backend termini inaspettatamente.

Le possibili cause sono numerose:

- un errore nel codice;
- memoria insufficiente;
- un aggiornamento del sistema operativo;
- un arresto improvviso del server.

L'applicazione smette immediatamente di rispondere.

Uno sviluppatore o un amministratore dovrà intervenire manualmente per ripristinare il servizio.

In un ambiente di produzione questo comportamento non è accettabile.

Il terzo problema

BookStore continua a crescere.

Ora il team desidera separare gli ambienti.

Sono necessari:

- Development
- Staging
- Production

Ogni ambiente possiede configurazioni differenti.

Database differenti.

Credenziali differenti.

Versioni differenti dell'applicazione.

Gestire tutto questo esclusivamente tramite Docker Compose diventa rapidamente complicato.

Il quarto problema

Ogni nuova versione del backend richiede il riavvio del container.

Durante questo intervallo il servizio non è disponibile.

Gli utenti ricevono errori.

Per applicazioni aziendali questo rappresenta un problema molto serio.

Idealmente vorremmo aggiornare il software senza interrompere il servizio.

Fermiamoci un momento

Osserva attentamente i problemi appena descritti.

Nessuno di essi riguarda i container.

I container funzionano perfettamente.

Il problema riguarda la loro **gestione**.

Non abbiamo bisogno di una nuova tecnologia per creare container.

Abbiamo bisogno di una tecnologia capace di gestirli automaticamente.

È proprio da questa esigenza che nasce Kubernetes.

Best Practice

Docker e Kubernetes non sono tecnologie concorrenti.

Docker crea ed esegue container.

Kubernetes li orchestra.

Nella maggior parte dei progetti moderni vengono utilizzati insieme.

Diagramma – Dalla containerizzazione all'orchestrazione

Figura 2.1 – Docker si occupa dei container; Kubernetes si occupa dell'intera applicazione.

Cosa vedremo nel prossimo paragrafo

Ora che abbiamo compreso il problema, possiamo introdurre la soluzione.

Nel prossimo paragrafo scopriremo che cosa significa realmente **orchestrare** un'applicazione containerizzata e perché Kubernetes è diventato lo standard de facto per questo tipo di attività.

2.2 Cos'è l'orchestrazione

Nel paragrafo precedente abbiamo analizzato alcuni problemi che emergono quando un'applicazione cresce.

Vale la pena osservarli nuovamente.

- Il backend può terminare improvvisamente.
- Il numero di utenti può aumentare rapidamente.
- È necessario distribuire nuove versioni dell'applicazione senza interrompere il servizio.
- Gli ambienti di sviluppo, test e produzione devono rimanere separati.
- L'applicazione deve continuare a funzionare anche in presenza di guasti.

Nessuno di questi problemi riguarda direttamente il codice sorgente.

Il frontend continua a funzionare.

Il backend continua a eseguire le proprie operazioni.

Il database continua a memorizzare i dati.

Il problema riguarda la gestione dell'intera applicazione.

Per comprendere questo concetto possiamo utilizzare un esempio molto semplice.

Un'orchestra senza direttore

Immagina un'orchestra composta da cinquanta musicisti.

Ogni musicista conosce perfettamente il proprio strumento.

Il violinista sa suonare il violino.

Il pianista sa suonare il pianoforte.

Il percussionista conosce perfettamente il proprio ritmo.

Ora immagina di eliminare il direttore d'orchestra.

Ogni musicista continuerà a suonare.

Probabilmente anche molto bene.

Il problema è che nessuno coordinerà più l'esecuzione.

Ogni musicista inizierà quando preferisce.

Cambierà il tempo.

Cambierà il volume.

Cambierà il ritmo.

L'orchestra non smetterà di suonare.

Semplicemente non suonerà più insieme.

I container sono come i musicisti

Docker svolge perfettamente il proprio lavoro.

Sa creare un container.

Sa avviarlo.

Sa arrestarlo.

Sa eliminarlo.

Ogni container è perfettamente autonomo.

Ma Docker non possiede una visione completa dell'intera applicazione.

Non decide quante copie del backend debbano essere in esecuzione.

Non controlla se un container è terminato.

Non distribuisce automaticamente il traffico tra più istanze.

Non sceglie su quale server eseguire un'applicazione.

Per fare tutto questo serve un componente superiore.

Esattamente come il direttore d'orchestra.

Che cosa significa orchestrare?

Orchestrare significa coordinare automaticamente tutti i componenti di un'applicazione.

In altre parole, un orchestratore prende decisioni che, altrimenti, dovrebbero essere prese manualmente dagli amministratori di sistema.

Per esempio.

Se il backend si interrompe, l'orchestratore lo riavvia.

Se il numero di utenti aumenta, l'orchestratore può creare nuove istanze.

Se un server smette di funzionare, l'orchestratore sposta automaticamente l'applicazione su un altro nodo disponibile.

Lo sviluppatore non deve intervenire.

L'obiettivo è mantenere il sistema sempre nello stato desiderato.

Dal modello imperativo al modello dichiarativo

Per molti sviluppatori questa rappresenta la differenza più difficile da comprendere.

Con Docker siamo abituati a impartire istruzioni.

```
Crea un container.
```

```
Avvia il container.
```

```
Ferma il container.
```

```
Elimina il container.
```

Questo approccio viene definito **imperativo**.

Lo sviluppatore descrive esattamente quali operazioni devono essere eseguite.

Kubernetes introduce un modo completamente diverso di lavorare.

Lo sviluppatore non descrive più i singoli passaggi.

Descrive soltanto il risultato finale.

Per esempio.

```
replicas: 3
```

Questa semplice proprietà non dice a Kubernetes come creare tre Pod.

Dice semplicemente:

Voglio sempre tre repliche del backend.

Come raggiungere questo obiettivo è responsabilità del cluster.

Questo approccio prende il nome di **modello dichiarativo**.

Perché il modello dichiarativo è così importante?

Immaginiamo che uno dei tre Pod venga eliminato accidentalmente.

Con un approccio imperativo qualcuno dovrebbe eseguire nuovamente i comandi necessari per ricrearlo.

Con Kubernetes non è necessario.

Il cluster osserva continuamente lo stato reale dell'applicazione.

Se rileva una differenza rispetto allo stato dichiarato, interviene automaticamente.

Questo meccanismo è chiamato **reconciliation loop**.

È uno dei principi fondamentali di Kubernetes.

Lo ritroveremo continuamente anche nei capitoli dedicati a OpenShift e ad Argo CD.

Il ruolo dello sviluppatore cambia

Questo è probabilmente il cambiamento più importante introdotto da Kubernetes.

Lo sviluppatore non gestisce più direttamente i container.

Definisce l'infrastruttura attraverso file YAML.

Quei file diventano una descrizione completa dell'applicazione.

Potremmo quasi considerarli una forma di codice.

Per questo motivo si parla spesso di **Infrastructure as Code (IaC)**.

L'infrastruttura viene descritta, versionata e revisionata esattamente come qualsiasi altro progetto software.

Best Practice

Evita di pensare a Kubernetes come a uno strumento che esegue comandi.

Consideralo invece come un sistema che lavora continuamente per mantenere il cluster nello stato dichiarato dai tuoi manifest YAML.

Diagramma – Dal modello imperativo al modello dichiarativo

Figura 2.2 – Nel modello imperativo descriviamo le operazioni da eseguire; nel modello dichiarativo descriviamo il risultato desiderato.

Un'anticipazione

Nel prossimo paragrafo installeremo un cluster Kubernetes locale.

Da quel momento ogni concetto verrà verificato attraverso laboratori pratici.

Non ci limiteremo a leggere i manifest YAML.

Li applicheremo.

Li modificheremo.

Li romperemo intenzionalmente.

Osserveremo il comportamento del cluster e impareremo a ragionare come Kubernetes.

Solo dopo aver acquisito questa mentalità inizieremo a utilizzare OpenShift.

2.3 Prepariamo il laboratorio

Prima di iniziare a lavorare con Kubernetes dobbiamo predisporre un ambiente di sviluppo locale.

Uno degli obiettivi principali di questo libro è permettere al lettore di sperimentare ogni argomento senza sostenere costi e senza utilizzare servizi cloud.

Per questo motivo tutti gli esempi verranno eseguiti direttamente sul computer dello sviluppatore.

L'unico requisito è disporre di una macchina sufficientemente potente per eseguire alcuni container contemporaneamente.

L'infrastruttura che realizzeremo durante il libro sarà composta esclusivamente da software open source.

L'ambiente di lavoro

Durante il resto del libro utilizzeremo il seguente stack.

Componente	Scopo
Git	Versionamento del codice
Docker Desktop oppure Docker Engine	Runtime dei container
kubectl	Client Kubernetes
Kind oppure Minikube	Cluster Kubernetes locale
Visual Studio Code	Editor
GitHub	Repository Git

Nei capitoli successivi aggiungeremo progressivamente:

- Helm
- OpenShift Local
- Tekton
- Argo CD

In questo modo ogni nuova tecnologia si appoggerà a quelle già apprese.

Perché un cluster locale?

Molti corsi iniziano direttamente utilizzando un cluster cloud.

Questa scelta presenta però alcuni svantaggi.

Prima di tutto introduce un costo.

Inoltre richiede una connessione Internet stabile.

Infine costringe il lettore ad imparare contemporaneamente Kubernetes e la piattaforma cloud scelta.

Noi adotteremo un approccio differente.

L'intero percorso verrà svolto su localhost.

Questo significa che potrai:

- sperimentare liberamente;
- rompere il cluster senza conseguenze;
- ricrearlo in pochi minuti;
- lavorare anche senza connessione Internet.

Una volta acquisiti i concetti fondamentali sarà molto semplice migrare verso OpenShift.

Kind o Minikube?

Esistono numerosi strumenti per creare un cluster Kubernetes locale.

I due più diffusi sono:

- Kind
- Minikube

Entrambi permettono di creare un cluster completo direttamente sul proprio computer.

Per il resto del libro utilizzeremo **Kind**.

La scelta è dovuta a tre motivi principali.

Il primo è la semplicità.

Kind crea un cluster utilizzando normali container Docker.

Il secondo riguarda la velocità.

L'avvio richiede generalmente meno di un minuto.

Infine, Kind si integra molto bene con gli strumenti che introdurremo nei capitoli successivi.

Questo non significa che Minikube sia una scelta sbagliata.

Tutti gli esempi presenti nel libro funzionano anche con Minikube.

Installazione di kubectl

Il primo componente da installare è `kubectl`.

Si tratta del client ufficiale di Kubernetes.

Ogni comando che eseguiremo durante il libro passerà attraverso questo strumento.

Verifichiamo che sia installato.

```
kubectl version --client
```

L'output dovrebbe essere simile al seguente.

```
Client Version: v1.xx.x  
Kustomize Version: v5.x.x
```

Se il comando restituisce un errore significa che kubectl non è ancora disponibile nel PATH del sistema operativo.

Creazione del cluster

Una volta installato Kind possiamo creare il nostro primo cluster.

```
kind create cluster --name bookstore
```

Il processo richiede generalmente meno di un minuto.

Al termine possiamo verificare che tutto sia andato a buon fine.

```
kubectl cluster-info
```

Output.

```
Kubernetes control plane is running...  
CoreDNS is running...
```

Il cluster è operativo.

Verifichiamo il nodo

Ogni cluster Kubernetes è composto da uno o più nodi.

Nel nostro laboratorio utilizzeremo un singolo nodo.

Controlliamo il suo stato.

```
kubectl get nodes
```

Output.

NAME	STATUS	ROLES	AGE
bookstore-control-plane	Ready		

La colonna **STATUS** rappresenta una delle informazioni più importanti.

Se il valore è **Ready**, Kubernetes è in grado di distribuire nuove applicazioni.

Il primo comando davvero utile

Durante tutto il libro utilizzeremo continuamente il seguente comando.

```
kubectl get all
```

Questo comando mostra una panoramica delle principali risorse presenti nel Namespace corrente.

In questo momento il cluster è praticamente vuoto.

Ed è proprio da qui che inizieremo.

Come è fatto il nostro cluster?

Anche se utilizziamo un solo nodo, possiamo già immaginare l'architettura che costruiremo nei prossimi capitoli.

Figura 2.3 – Architettura del cluster locale utilizzato durante il libro.

Nel corso del capitolo aggiungeremo progressivamente:

- Namespace;
- Deployment;
- Service;
- Pod.

Alla fine otterremo una piattaforma completa capace di eseguire l'applicazione BookStore.

Laboratorio 2.1

Prima di proseguire verifica che il tuo ambiente sia pronto.

Esegui i seguenti comandi.

```
kubectl version --client  
kubectl cluster-info  
kubectl get nodes  
kubectl get all
```

Se tutti i comandi restituiscono il risultato atteso puoi passare al paragrafo successivo.

In caso contrario interrompi la lettura e risolvi il problema.

Tutti i laboratori successivi presuppongono che il cluster sia correttamente configurato.

Molti sviluppatori iniziano a creare manifest YAML senza aver verificato che il cluster sia realmente operativo.

Prima di distribuire qualsiasi applicazione controlla sempre:

- il nodo sia **Ready**;
 - `kubectl` riesca a comunicare con il cluster;
 - non siano presenti errori durante l'avvio del Control Plane.
-

Cosa vedremo nel prossimo paragrafo

Adesso il laboratorio è pronto.

È arrivato il momento di distribuire la prima applicazione.

Nel prossimo paragrafo creeremo il nostro primo **Pod**.

Per la prima volta Kubernetes interpreterà un manifest YAML scritto da noi e creerà automaticamente una risorsa all'interno del cluster.

Da questo momento inizieremo davvero a utilizzare Kubernetes.

2.4 Il Pod

Abbiamo finalmente un cluster Kubernetes funzionante.

Possiamo comunicare con il Control Plane.

Possiamo verificare lo stato dei nodi.

Siamo pronti a distribuire la nostra prima applicazione.

Prima di scrivere qualsiasi file YAML è però necessario comprendere quale sia l'unità fondamentale di Kubernetes.

Molti sviluppatori rispondono immediatamente:

"Il container."

In realtà la risposta è diversa.

L'unità minima distribuibile in Kubernetes non è il container.

È il **Pod**.

Comprendere questo concetto è fondamentale, perché tutto ciò che realizzeremo nel resto del libro ruoterà attorno ai Pod.

Perché Kubernetes non utilizza direttamente i container?

Questa è probabilmente la domanda più importante di questo capitolo.

Se Docker è già in grado di creare ed eseguire container, perché Kubernetes introduce un nuovo livello di astrazione?

La risposta è semplice.

Un'applicazione reale raramente è composta da un solo processo.

Molto spesso è necessario affiancare al processo principale uno o più componenti di supporto.

Ad esempio:

- un proxy;
- un collector di log;
- un agente di monitoraggio;
- un container dedicato alla configurazione iniziale.

Questi componenti devono condividere:

- la rete;
- lo spazio dei nomi;
- alcuni volumi.

Gestire manualmente queste dipendenze sarebbe estremamente complicato.

Il Pod nasce proprio per risolvere questo problema.

Cos'è un Pod?

Un Pod è il più piccolo oggetto distribuibile all'interno di Kubernetes.

Può contenere uno o più container che collaborano tra loro.

Tutti i container presenti nello stesso Pod condividono:

- lo stesso indirizzo IP;
- lo stesso namespace di rete;
- gli stessi volumi dichiarati nel manifest;
- parte del ciclo di vita.

Nella maggior parte delle applicazioni ogni Pod contiene un solo container.

È esattamente quello che faremo nel progetto BookStore.

Più avanti nel libro incontreremo esempi nei quali un Pod ospiterà più container, ma per il momento manterremo l'architettura il più semplice possibile.

Un'analogia

Immagina un appartamento.

All'interno possono vivere una o più persone.

Condividono:

- l'indirizzo;
- l'impianto elettrico;
- la connessione Internet;
- la cucina.

Il Pod rappresenta l'appartamento.

I container rappresentano gli occupanti.

L'indirizzo appartiene al Pod.

Non ai singoli container.

Questa differenza è molto importante.

Il nostro primo manifest

Creiamo una nuova cartella.

```
bookstore/  
└─ kubernetes/  
    └─ pods/
```

All'interno della directory creiamo il file.

```
backend-pod.yaml
```

Inseriamo il seguente contenuto.

```
apiVersion: v1  
  
kind: Pod  
  
metadata:  
  name: bookstore-backend  
  
spec:  
  containers:  
    - name: backend  
      image: bookstore/backend:1.0  
      ports:  
        - containerPort: 8080
```

Questo è il primo manifest Kubernetes che scriviamo.

Può sembrare molto semplice.

In realtà descrive completamente una nuova risorsa del cluster.

Analizziamo il manifest

Ogni manifest Kubernetes è composto da alcune sezioni fondamentali.

Le incontreremo continuamente durante tutto il libro.

Per questo motivo vale la pena comprenderle fin da subito.

apiVersion

```
apiVersion: v1
```

Indica la versione dell'API utilizzata per interpretare il manifest.

Ogni risorsa Kubernetes appartiene a un gruppo API.

Per i Pod utilizziamo la versione **v1**.

kind

```
kind: Pod
```

Specifica il tipo di risorsa che desideriamo creare.

In questo caso stiamo chiedendo al cluster di creare un Pod.

Nei prossimi paragrafi vedremo altri tipi di risorse, come Deployment e Service.

metadata

```
metadata:  
  name: bookstore-backend
```

La sezione `metadata` contiene le informazioni descrittive della risorsa.

In questo caso assegniamo al Pod il nome `bookstore-backend`.

Questo nome dovrà essere univoco all'interno del Namespace.

spec

La sezione `spec` rappresenta il cuore del manifest.

Qui descriviamo lo stato desiderato del Pod.

Nel nostro caso stiamo dichiarando:

- un container;
- il nome del container;
- l'immagine Docker;
- la porta esposta.

Ogni volta che Kubernetes leggerà questo manifest tenterà di mantenere il Pod nello stato descritto.

Cosa succede quando applichiamo il manifest?

Salviamo il file.

Ora eseguiamo.

```
kubectl apply -f backend-pod.yaml
```

L'output sarà simile al seguente.

```
pod/bookstore-backend created
```

Questo semplice messaggio nasconde numerose operazioni.

Il cluster ha:

1. letto il manifest;
2. verificato la validità della sintassi;
3. registrato la nuova risorsa;
4. scelto il nodo sul quale eseguire il Pod;
5. scaricato l'immagine del container;
6. avviato il processo.

Tutto questo è avvenuto automaticamente.

Verifichiamo il risultato

Controlliamo che il Pod sia effettivamente in esecuzione.

```
kubectl get pods
```

Output.

NAME	READY	STATUS	RESTARTS	AGE
bookstore-backend	1/1	Running	0	12s

La colonna **STATUS** indica lo stato corrente del Pod.

Nel nostro caso il valore è `Running`.

Significa che il container è stato creato correttamente ed è in esecuzione.

Osserviamo il Pod nel dettaglio

Il comando `get` fornisce una panoramica molto sintetica.

Per ottenere maggiori informazioni possiamo utilizzare:

```
kubectl describe pod bookstore-backend
```

Tra le informazioni visualizzate troveremo:

- nodo utilizzato;
- indirizzo IP;
- immagine Docker;
- eventi;
- stato del container.

Imparare a leggere l'output di `describe` è una delle competenze più importanti per chi lavora con Kubernetes.

Nei capitoli successivi lo utilizzeremo continuamente per individuare eventuali problemi.

Best Practice

Quando una risorsa non si comporta come previsto, evita di modificare immediatamente il manifest.

Il primo passo dovrebbe essere sempre osservare lo stato del cluster attraverso i comandi:

- `kubectl get`
- `kubectl describe`
- `kubectl logs`

Nella maggior parte dei casi la risposta è già disponibile all'interno del cluster.

Laboratorio 2.2 – Creiamo il primo Pod

È arrivato il momento di distribuire la nostra prima applicazione sul cluster.

Lo faremo partendo dal manifest creato nel paragrafo precedente.

Verifica di trovarti nella cartella del progetto.

```
cd bookstore/kubernetes/pods
```

Applica il manifest.

```
kubectl apply -f backend-pod.yaml
```

L'output dovrebbe essere simile al seguente.

```
pod/bookstore-backend created
```

Il cluster ha accettato la richiesta e ha iniziato il processo di creazione del Pod.

Controlliamo lo stato.

```
kubectl get pods
```

Dopo alcuni secondi dovresti ottenere un risultato simile.

NAME	READY	STATUS	RESTARTS	AGE
bookstore-backend	1/1	Running	0	18s

Il Pod è stato creato correttamente.

Cosa significano queste colonne?

L'output di `kubectl get pods` contiene molte informazioni utili.

Analizziamo le più importanti.

Colonna	Significato
NAME	Nome del Pod
READY	Numero di container pronti rispetto al totale
STATUS	Stato corrente del Pod
RESTARTS	Numero di riavvii del container
AGE	Tempo trascorso dalla creazione

Nei prossimi capitoli controlleremo continuamente questi valori.

Imparare a leggerli velocemente permette di individuare molti problemi senza utilizzare strumenti aggiuntivi.

Lo stato di un Pod

Un Pod attraversa diversi stati durante il proprio ciclo di vita.

I più comuni sono:

Stato	Significato
Pending	Kubernetes sta preparando il Pod
Running	Il Pod è operativo
Completed	Il processo è terminato correttamente
CrashLoopBackOff	Il container continua a terminare in errore
Error	Errore durante l'esecuzione
ImagePullBackOff	Impossibile scaricare l'immagine Docker

Non è necessario memorizzarli tutti.

Nel corso dei laboratori li incontreremo naturalmente.

Osserviamo il Pod

Visualizziamo tutte le informazioni disponibili.

```
kubectl describe pod bookstore-backend
```

L'output è molto lungo.

Le sezioni più importanti sono:

- Nome
- Namespace
- Node
- IP
- Containers
- Events

Particolare attenzione va dedicata agli **Events**.

Questa sezione racconta tutto ciò che Kubernetes ha fatto per creare il Pod.

Ad esempio:

- il Pod è stato schedulato;
- l'immagine è stata scaricata;
- il container è stato creato;
- il container è stato avviato.

Quando qualcosa non funziona, gli Events rappresentano quasi sempre il primo punto da cui iniziare l'analisi.

Leggiamo i log

Anche se il Pod risulta nello stato **Running**, questo non garantisce che l'applicazione stia funzionando correttamente.

Per verificare cosa sta facendo il processo principale utilizziamo.

```
kubectl logs bookstore-backend
```

Il comando mostra l'output standard del container.

Nel caso di una applicazione Spring Boot vedremo i messaggi di avvio del framework.

Durante le attività di troubleshooting questo comando diventerà uno degli strumenti più utilizzati.

Esperimento 2.1 – Eliminiamo il Pod

Adesso eseguiamo un esperimento.

Eliminiamo il Pod appena creato.

```
kubectl delete pod bookstore-backend
```

Controlliamo nuovamente il cluster.

```
kubectl get pods
```

Il risultato potrebbe sorprenderti.

```
No resources found.
```

Il Pod non esiste più.

Ed è esattamente ciò che ci aspettavamo.

Perché il Pod non viene ricreato?

Molti lettori arrivano a Kubernetes dopo aver studiato Docker Compose.

A questo punto potrebbero aspettarsi che il cluster ricrei automaticamente il Pod.

In realtà non succede.

Ed è corretto che sia così.

Noi abbiamo chiesto a Kubernetes di creare **un Pod**.

Non gli abbiamo chiesto di mantenerlo sempre disponibile.

Questa responsabilità appartiene a un'altra risorsa.

Il Deployment.

Il limite dei Pod

I Pod sono fondamentali.

Tuttavia presentano alcuni limiti importanti.

Se vengono eliminati, non vengono ricreati automaticamente.

Non possono essere scalati.

Non gestiscono aggiornamenti progressivi.

Non permettono rollback.

Per questo motivo, nei progetti reali, difficilmente troverai Pod creati direttamente.

Quasi sempre vengono creati attraverso un Deployment.

Best Practice

Considera il Pod come l'unità minima di esecuzione.

Considera il Deployment come il vero punto di ingresso per distribuire un'applicazione.

Durante lo sviluppo può essere utile creare Pod manualmente per comprendere il loro funzionamento.

In produzione utilizzerai quasi esclusivamente Deployment.

Diagramma – Il ciclo di vita di un Pod

Figura 2.4 – Stati principali attraversati da un Pod durante la sua esecuzione.

Cosa vedremo nel prossimo paragrafo

Abbiamo imparato a creare un Pod.

Abbiamo osservato come viene eseguito.

Abbiamo imparato a leggere il suo stato.

Abbiamo consultato i log.

Infine abbiamo scoperto il suo principale limite.

Nel prossimo paragrafo introdurremo il **Deployment**.

Sarà il primo componente di Kubernetes capace di mantenere automaticamente lo stato desiderato dell'applicazione.

Con il Deployment inizieremo finalmente a comprendere il vero modello dichiarativo di Kubernetes.

2.5 Il Deployment

Nel paragrafo precedente abbiamo creato il nostro primo Pod.

Abbiamo imparato a:

- distribuirlo;
- verificarne lo stato;
- leggere i log;
- eliminarlo.

L'ultimo esperimento ci ha però mostrato un limite importante.

Quando il Pod viene eliminato, Kubernetes non lo ricrea.

Perché?

La risposta è semplice.

Noi abbiamo chiesto al cluster di creare un Pod.

Nulla di più.

Una volta soddisfatta la richiesta, Kubernetes considera il proprio lavoro terminato.

Questo comportamento è perfettamente corretto.

Il problema è che, in un ambiente reale, quasi nessuna applicazione può permettersi di rimanere offline dopo l'eliminazione di un singolo Pod.

Serve quindi un componente che osservi continuamente il cluster e mantenga l'applicazione nello stato desiderato.

Questo componente prende il nome di **Deployment**.

Pensare in termini di stato

Il Deployment introduce un concetto completamente nuovo.

Non descrive un singolo Pod.

Descrive il risultato che vogliamo ottenere.

Per esempio.

Voglio sempre una copia del backend.

Oppure.

Voglio tre copie del backend.

Oppure ancora.

Voglio aggiornare tutti i Pod alla versione 2.0 senza interrompere il servizio.

Lo sviluppatore smette quindi di impartire ordini.

Inizia invece a descrivere un obiettivo.

Il cluster farà il resto.

Il nostro primo Deployment

Creiamo una nuova cartella.

```
bookstore/  
└─ kubernetes/  
    └─ deployments/
```

All'interno creiamo il file.

```
backend-deployment.yaml
```

Inseriamo il seguente contenuto.

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: bookstore-backend  
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: bookstore-backend  
  template:  
    metadata:  
      labels:  
        app: bookstore-backend  
    spec:  
      containers:  
        - name: backend  
          image: bookstore/backend:1.0  
          ports:  
            - containerPort: 8080
```

A prima vista questo manifest sembra molto più lungo rispetto a quello del Pod.

In realtà contiene gli stessi elementi, più alcune informazioni aggiuntive necessarie al Deployment per gestire automaticamente i Pod.

Analizziamo il manifest

Prima di applicarlo, osserviamo le nuove sezioni introdotte.

replicas

```
replicas: 1
```

Questa proprietà rappresenta il numero di Pod che Kubernetes dovrà mantenere attivi.

Non stiamo creando un Pod.

Stiamo dichiarando il numero di Pod desiderati.

Questa differenza è fondamentale.

selector

```
selector:  
  matchLabels:  
    app: bookstore-backend
```

Il Deployment deve sapere quali Pod appartengono all'applicazione.

Per questo motivo utilizza un selector.

Il selector ricerca tutti i Pod che possiedono la Label.

```
app=bookstore-backend
```

Ogni Pod trovato verrà considerato parte del Deployment.

template

La sezione `template` descrive il modello utilizzato per creare nuovi Pod.

Possiamo considerarla una specie di stampo.

Ogni volta che il Deployment avrà bisogno di una nuova replica utilizzerà proprio questo template.

Per questo motivo il manifest del Pod compare nuovamente all'interno del Deployment.

Applichiamo il manifest

Salviamo il file.

Ora distribuiamo il Deployment.

```
kubectl apply -f backend-deployment.yaml
```

Output.

```
deployment.apps/bookstore-backend created
```

Controlliamo il risultato.

```
kubectl get deployments
```

Output.

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
bookstore-backend	1/1	1	1	20s

Adesso osserviamo i Pod.

```
kubectl get pods
```

Output.

NAME	READY
bookstore-backend-6d5cfb89df-lx8pw	1/1

Il nome del Pod è diverso rispetto a prima.

Non è più semplicemente `bookstore-backend`.

Kubernetes aggiunge automaticamente alcuni identificatori che permettono di distinguere le varie repliche create dal Deployment.

Cosa è successo?

Quando abbiamo applicato il manifest non è stato creato direttamente un Pod.

È stato creato un Deployment.

Il Deployment ha quindi creato un ReplicaSet.

Il ReplicaSet ha infine creato il Pod.

Per il momento non approfondiremo il ruolo del ReplicaSet.

È sufficiente sapere che rappresenta un componente interno utilizzato dal Deployment.

Lo incontreremo nuovamente più avanti.

Diagramma – Come nasce un Pod

Figura 2.5 – Il Deployment non crea direttamente i Pod. Utilizza un ReplicaSet come componente intermedio.

Un nuovo modo di osservare il cluster

Fino a questo momento abbiamo utilizzato principalmente.

```
kubectl get pods
```

Da ora in avanti inizieremo a utilizzare molto spesso anche.

```
kubectl get deployments
```

e

```
kubectl get replicaset
```

Questi tre comandi permettono di osservare l'intero ciclo di vita dell'applicazione.

Nei prossimi laboratori diventeranno strumenti indispensabili.

Come fa il Service a trovare i Pod?

Nel paragrafo precedente abbiamo creato il nostro primo Service.

Il manifest conteneva una proprietà che, a prima vista, potrebbe sembrare poco importante.

```
selector:
```

```
  app: bookstore-backend
```

In realtà questa è una delle caratteristiche più potenti di Kubernetes.

Il Service non contiene l'elenco dei Pod.

Non conosce i loro nomi.

Non conosce i loro indirizzi IP.

Conosce soltanto una Label.

Ogni Pod che possiede quella Label entrerà automaticamente a far parte del Service.

Questa scelta rende l'intera architettura estremamente dinamica.

Non importa quante repliche verranno create.

Non importa su quale nodo verranno eseguite.

Non importa se un Pod verrà eliminato e ricreato.

Finché la Label rimane la stessa, il Service continuerà a funzionare senza alcuna modifica.

Il ruolo delle Labels

Le Labels rappresentano uno dei concetti fondamentali di Kubernetes.

Sono semplici coppie chiave/valore associate alle risorse del cluster.

Per esempio.

```
labels:
```

```
  app: bookstore-backend
```

```
  tier: backend
```

```
  environment: development
```

Una stessa risorsa può possedere numerose Labels.

Ogni componente del cluster può utilizzarle per individuare rapidamente gli oggetti di interesse.

Nel nostro progetto il Deployment assegna automaticamente la Label.

```
app: bookstore-backend
```

Il Service ricerca esattamente quella stessa Label.

È così che nasce il collegamento tra le due risorse.

Diagramma – Deployment e Service

Figura 2.7 – Il Service individua automaticamente tutti i Pod che possiedono la Label corretta.

Verifichiamo gli Endpoint

Ogni Service mantiene un elenco dei Pod verso cui può inoltrare il traffico.

Questo elenco prende il nome di **Endpoint**.

Visualizziamolo.

```
kubectl get endpoints
```

Output.

NAME	ENDPOINTS
bookstore-backend-service	10.244.0.12:8080

Se aumentiamo il numero delle repliche.

```
kubectl scale deployment bookstore-backend \
--replicas=3
```

e controlliamo nuovamente.

```
kubectl get endpoints
```

otterremo un risultato simile.

NAME	ENDPOINTS
bookstore-backend-service	10.244.0.12:8080, 10.244.0.18:8080, 10.244.0.21:8080

Gli Endpoint vengono aggiornati automaticamente.

Non abbiamo modificato il Service.

Non abbiamo modificato il Deployment.

Kubernetes ha semplicemente osservato che sono comparsi nuovi Pod con la Label corretta.

Bilanciamento del traffico

A questo punto il Service possiede tre Pod disponibili.

Che cosa succede quando il frontend invia una richiesta?

Il frontend non sceglie alcun Pod.

Invia semplicemente una richiesta al Service.

Sarà Kubernetes a distribuirla verso una delle repliche disponibili.

Questo comportamento prende il nome di **load balancing**.

Per applicazioni stateless rappresenta uno dei principali vantaggi dell'utilizzo di Kubernetes.

Lo sviluppatore non deve implementare alcuna logica di bilanciamento.

È il cluster ad occuparsene.

DNS interno

Un altro vantaggio dei Service riguarda il DNS.

Ogni Service viene automaticamente registrato nel sistema DNS interno del cluster.

Questo significa che il frontend non dovrà utilizzare un indirizzo IP.

Potrà semplicemente utilizzare il nome del Service.

Nel nostro caso.

```
bookstore-backend-service
```

Ogni richiesta indirizzata a questo nome verrà automaticamente inoltrata verso uno dei Pod disponibili.

Questo meccanismo rende l'applicazione completamente indipendente dagli indirizzi IP.

È uno dei motivi per cui Kubernetes può sostituire continuamente i Pod senza interrompere il servizio.

Modifichiamo il frontend

Immaginiamo che il frontend React debba contattare il backend.

Un approccio errato sarebbe.

```
http://10.244.0.12:8080
```

Questo indirizzo smetterebbe di funzionare non appena il Pod venisse sostituito.

L'approccio corretto è utilizzare il nome del Service.

```
http://bookstore-backend-service:8080
```


In questo modo il frontend continuerà a funzionare anche se il Deployment creerà nuove repliche o sostituirà completamente i Pod esistenti.

Esperimento 2.5 – Il Service continua a funzionare

Lasciamo il frontend collegato al Service.

Adesso eliminiamo uno dei Pod.

```
kubect1 delete pod bookstore-backend-xxxxxxx
```

Attendi alcuni secondi.

Il Deployment creerà automaticamente una nuova replica.

Il Service aggiornerà i propri Endpoint.

Il frontend continuerà a comunicare con il backend senza alcuna modifica.

Per l'utente finale non cambierà assolutamente nulla.

Questo è uno dei primi esempi concreti di alta disponibilità offerta da Kubernetes.

Best Practice

Le applicazioni non dovrebbero mai comunicare utilizzando gli indirizzi IP dei Pod.

Utilizza sempre il nome del Service.

In questo modo la tua applicazione rimarrà indipendente dal ciclo di vita dei Pod e potrà beneficiare automaticamente del bilanciamento del traffico e della riconciliazione del cluster.

Cosa abbiamo imparato?

Con l'introduzione del Service il nostro progetto BookStore inizia ad assumere la forma di una vera applicazione distribuita.

Abbiamo separato le responsabilità.

Il Deployment gestisce il ciclo di vita dei Pod.

Il Service gestisce la comunicazione verso i Pod.

Il frontend conosce soltanto il Service.

Non conosce il numero delle repliche.

Non conosce gli indirizzi IP.

Non conosce il nodo sul quale il backend è in esecuzione.

Questa separazione rappresenta uno dei principi fondamentali dell'architettura Kubernetes.

Nel prossimo paragrafo introdurremo i **Namespace**.

Scopriremo come organizzare logicamente tutte le risorse del cluster e come preparare il terreno per la gestione di ambienti differenti come Development, Staging e Production.

2.6 Il Service

Nel laboratorio precedente abbiamo creato un Deployment.

Successivamente abbiamo aumentato il numero delle repliche.

Infine abbiamo eliminato alcuni Pod per osservare il comportamento del cluster.

Tutto ha funzionato perfettamente.

C'è però un problema.

Un problema molto serio.

Un esperimento

Visualizziamo i Pod.

```
kubectl get pods -o wide
```

Output.

NAME	IP
bookstore-backend-6d5cfb89df-lx8pw	10.244.0.12
bookstore-backend-7849fd56db-kd2xw	10.244.0.18
bookstore-backend-8a3bc6d774-rp91h	10.244.0.21

Ogni Pod possiede un proprio indirizzo IP.

Supponiamo adesso che il frontend comunichi direttamente con

```
10.244.0.12
```

Funzionerebbe?

Sì.

Almeno fino a quando quel Pod rimane in esecuzione.

Eliminiamo il Pod

Ripetiamo un esperimento già visto.

```
kubectl delete pod bookstore-backend-6d5cfb89df-lx8pw
```

Attendiamo alcuni secondi.

Visualizziamo nuovamente i Pod.

```
kubectl get pods -o wide
```

Output.

NAME	IP
bookstore-backend-91c88df7d8-rq82m	10.244.0.27

Hai notato?

Il Pod è stato ricreato.

Ma l'indirizzo IP è cambiato.

Il frontend continuerebbe a cercare il vecchio indirizzo.

L'applicazione smetterebbe immediatamente di funzionare.

Il problema

Questo comportamento non rappresenta un errore.

È il funzionamento normale di Kubernetes.

I Pod sono risorse temporanee.

Nascono.

Vengono eliminati.

Vengono ricreati.

Possono cambiare nodo.

Possono cambiare indirizzo IP.

Per questo motivo Kubernetes considera gli indirizzi IP dei Pod come informazioni interne.

Le applicazioni non dovrebbero mai utilizzarli direttamente.

Serve quindi un indirizzo stabile.

Un indirizzo che rimanga invariato anche quando i Pod vengono sostituiti.

Ed è esattamente questo il compito del Service.

Cos'è un Service?

Un Service è una risorsa Kubernetes che fornisce un punto di accesso stabile verso uno o più Pod.

Il Service possiede:

- un nome;
- un indirizzo IP interno;
- una porta;
- un insieme di Pod associati.

L'applicazione comunica sempre con il Service.

Mai con i Pod.

Sarà Kubernetes a scegliere automaticamente quale replica utilizzare.

Un'analogia

Immagina un centralino telefonico.

I clienti non conoscono il numero personale di ogni operatore.

Conoscono soltanto il numero del centralino.

Ogni telefonata viene inoltrata all'operatore disponibile.

Il Service svolge esattamente questo ruolo.

I Pod possono cambiare.

Possono aumentare.

Possono diminuire.

Il Service rimane sempre lo stesso.

Il nostro primo Service

Creiamo una nuova cartella.

```
bookstore/  
└─ kubernetes/  
    └─ services/
```

All'interno creiamo il file.

```
backend-service.yaml
```

Inseriamo il seguente contenuto.

```
apiVersion: v1  
kind: Service  
metadata:  
  name: bookstore-backend-service  
spec:  
  selector:  
    app: bookstore-backend  
  ports:  
    - port: 8080
```

```
targetPort: 8080
type: ClusterIP
```

Il manifest è molto più semplice rispetto a quello del Deployment.

Il suo compito è esclusivamente quello di instradare il traffico verso i Pod corretti.

Analizziamo il manifest

La proprietà più importante è il selector.

```
selector:
  app: bookstore-backend
```

Il Service cerca automaticamente tutti i Pod che possiedono questa Label.

Ogni nuovo Pod creato dal Deployment entrerà immediatamente a far parte del Service.

Non sarà necessario modificare alcuna configurazione.

Questo rappresenta uno dei principali vantaggi dell'architettura Kubernetes.

ClusterIP

Nel manifest compare anche una proprietà.

```
type: ClusterIP
```

ClusterIP rappresenta il tipo di Service predefinito.

Il Service sarà raggiungibile esclusivamente dall'interno del cluster.

È esattamente ciò di cui abbiamo bisogno.

Il frontend e il backend comunicano infatti all'interno della stessa rete Kubernetes.

Più avanti nel libro vedremo altri tipi di Service, come NodePort e LoadBalancer.

Per il momento utilizzeremo sempre ClusterIP.

Applichiamo il manifest

Distribuiamo il Service.

```
kubectl apply -f backend-service.yaml
```

Visualizziamo le risorse.

```
kubectl get services
```

Output.

NAME	TYPE	CLUSTER-IP
bookstore-backend-service	ClusterIP	10.96.143.25

A differenza dei Pod, questo indirizzo rimarrà stabile.

Potremo utilizzarlo per tutta la vita dell'applicazione.

2.7 I Namespace

Fino a questo momento abbiamo lavorato all'interno del Namespace predefinito del cluster.

Per un laboratorio molto semplice questo approccio può andare bene.

Ma immaginiamo che BookStore inizi a crescere.

Il team di sviluppo aumenta.

Le applicazioni diventano numerose.

Sul cluster vengono distribuiti:

- BookStore
- Customer Portal
- CRM
- API Gateway
- Monitoring
- Logging

Tutte queste applicazioni convivono nello stesso cluster.

Come possiamo evitare che le loro risorse si mescolino?

Kubernetes risponde a questa esigenza introducendo i **Namespace**.

Cos'è un Namespace?

Un Namespace rappresenta una suddivisione logica del cluster.

Possiamo immaginarlo come una cartella.

Ogni cartella contiene le proprie risorse.

Ad esempio.

```
Cluster
├─ default
├─ bookstore
├─ monitoring
├─ logging
└─ argocd
```

Ogni Namespace possiede:

- Pod;
- Deployment;

- Service;
- ConfigMap;
- Secret;
- Job;
- CronJob.

Le risorse appartenenti a un Namespace sono isolate da quelle presenti negli altri Namespace.

Perché utilizzare i Namespace?

Supponiamo di distribuire due applicazioni.

Entrambe possiedono un Deployment chiamato.

```
backend
```

Senza Namespace questo non sarebbe possibile.

Il nome entrerebbe immediatamente in conflitto.

Con i Namespace, invece, possiamo avere.

```
bookstore/backend
```

```
crm/backend
```

Entrambi i Deployment convivono senza alcun problema.

Questo semplice meccanismo permette di ospitare decine o centinaia di applicazioni nello stesso cluster.

Namespace e ambienti

Uno degli utilizzi più comuni riguarda la separazione degli ambienti.

Per esempio.

```
development
```

```
staging
```

```
production
```

Ogni ambiente possiede:

- configurazioni differenti;
- database differenti;
- Secret differenti;
- immagini differenti.

Pur condividendo lo stesso cluster, le applicazioni rimangono logicamente separate.

È importante sottolineare che un Namespace **non rappresenta una macchina virtuale**.

Tutte le risorse continuano a essere eseguite sul medesimo cluster.

Il Namespace fornisce esclusivamente un isolamento logico.

Il Namespace di BookStore

Per il resto del libro utilizzeremo un Namespace dedicato.

Creiamo il file.

```
namespace.yaml
```

Inseriamo il seguente contenuto.

```
apiVersion: v1
kind: Namespace
metadata:
  name: bookstore
```

Distribuiamo il manifest.

```
kubectl apply -f namespace.yaml
```

Verifichiamo.

```
kubectl get namespaces
```

Output.

```
NAME
default
bookstore
kube-system
kube-public
```

Il Namespace è stato creato correttamente.

Distribuiamo il backend nel nuovo Namespace

Da questo momento tutte le risorse del progetto verranno distribuite all'interno del Namespace bookstore.

Per esempio.

```
kubectl apply \
```

```
-f backend-deployment.yaml \
-n bookstore
```

oppure, direttamente nel manifest.

```
metadata:
```

```
  namespace: bookstore
```

Entrambi gli approcci sono validi.

Nel resto del libro utilizzeremo il secondo.

In questo modo ogni manifest descriverà completamente la risorsa, senza dipendere dai parametri passati sulla riga di comando.

Verifichiamo il risultato

Visualizziamo i Pod.

```
kubectl get pods
```

Output.

```
No resources found.
```

Il comando non restituisce alcun risultato.

Molti sviluppatori pensano immediatamente che il Deployment non abbia funzionato.

In realtà il problema è diverso.

Stiamo osservando il Namespace sbagliato.

Ripetiamo il comando.

```
kubectl get pods -n bookstore
```

Output.

```
NAME
```

```
bookstore-backend-6d5cfb89df-lx8pw
```

Il Pod è presente.

Semplicemente appartiene a un Namespace differente.

Come evitare di specificare continuamente il Namespace

Durante i laboratori utilizzeremo molto spesso il Namespace `bookstore`.

Scrivere ogni volta.

```
-n bookstore
```

diventa rapidamente noioso.

Possiamo modificare il contesto corrente.

```
kubectl config set-context \
--current \
--namespace=bookstore
```

Da questo momento tutti i comandi verranno eseguiti automaticamente nel Namespace corretto.

Per verificare il risultato.

```
kubectl config view --minify
```

Tra le informazioni visualizzate comparirà il Namespace corrente.

Diagramma – Organizzazione del cluster

Figura 2.8 – I Namespace permettono di organizzare logicamente le applicazioni all'interno dello stesso cluster.

Best Practice

Crea sempre un Namespace dedicato per ogni applicazione.

Evita di distribuire applicazioni aziendali nel Namespace `default`.

Questa semplice abitudine renderà il cluster molto più semplice da amministrare.

Nei capitoli dedicati a OpenShift vedremo che il concetto di **Project** deriva direttamente dai Namespace Kubernetes.

Per questo motivo imparare a utilizzarli correttamente rappresenta un investimento che continuerà a dare risultati anche quando migreremo verso OpenShift.

Cosa vedremo nel prossimo paragrafo

Fino a questo momento abbiamo utilizzato Kubernetes come una "scatola nera".

Abbiamo scritto manifest YAML.

Abbiamo applicato le modifiche.

Il cluster ha creato automaticamente le risorse.

Ma cosa succede realmente quando eseguiamo:

```
kubectl apply -f backend-deployment.yaml
```

Nel prossimo paragrafo entreremo finalmente all'interno del cluster.

Seguiremo il percorso di una richiesta dal comando `kubectl` fino alla creazione del Pod.

Comprendere questo flusso ti permetterà di interpretare molto più facilmente il funzionamento di OpenShift, Helm, Tekton e Argo CD.

2.8 Come ragiona Kubernetes

Fino a questo momento abbiamo utilizzato Kubernetes attraverso un solo comando.

```
kubectl apply -f backend-deployment.yaml
```

Il Deployment è stato creato.

Il Pod è comparso.

L'applicazione ha iniziato a funzionare.

Tutto è sembrato quasi "magico".

Naturalmente non c'è nulla di magico.

Dietro quel semplice comando lavorano numerosi componenti che collaborano continuamente tra loro.

Comprendere questo flusso rappresenta uno dei passaggi più importanti dell'intero percorso di apprendimento.

Il viaggio di una richiesta

Immaginiamo di eseguire.

```
kubectl apply -f backend-deployment.yaml
```

Dal nostro punto di vista è un singolo comando.

Dal punto di vista del cluster, invece, inizia una lunga sequenza di operazioni.

Possiamo rappresentarla così.

```
kubectl
↓
API Server
↓
etcd
↓
Controller
↓
Scheduler
↓
kubelet
↓
```

Container Runtime

↓

Pod

Ogni componente svolge un ruolo ben preciso.

Analizziamoli uno alla volta.

1. kubectl

Il primo componente è il più semplice.

`kubectl` non appartiene realmente al cluster.

È semplicemente il programma che utilizziamo sul nostro computer.

Il suo unico compito consiste nell'inviare richieste al cluster Kubernetes.

Possiamo immaginarlo come un telecomando.

Il telecomando non accende la televisione.

Invia soltanto il comando.

Sarà poi la televisione ad eseguirlo.

Allo stesso modo, `kubectl` non crea alcun Pod.

Invia semplicemente una richiesta al cluster.

2. API Server

L'API Server rappresenta la porta di ingresso del cluster.

Qualsiasi richiesta passa obbligatoriamente da questo componente.

Non importa se la richiesta proviene da:

- `kubectl`;
- Helm;
- Argo CD;
- OpenShift;
- Tekton;
- Dashboard Kubernetes.

Tutto passa dall'API Server.

Per questo motivo viene spesso definito il cuore del Control Plane.

Quando riceve il manifest, l'API Server esegue numerosi controlli.

Verifica che:

- il file YAML sia valido;
- il tipo di risorsa esista;
- l'utente possieda le autorizzazioni necessarie;
- i dati siano coerenti.

Se uno di questi controlli fallisce, la richiesta viene rifiutata.

3. etcd

Una volta validata la richiesta, l'API Server registra il nuovo stato del cluster.

Queste informazioni vengono salvate in un database distribuito chiamato **etcd**.

È importante comprendere un aspetto.

etcd non contiene i container.

Non contiene i Pod.

Non contiene le immagini Docker.

Contiene esclusivamente la descrizione dello stato del cluster.

In altre parole.

Quando eseguiamo.

```
replicas: 3
```

quel valore viene memorizzato in etcd.

Da questo momento Kubernetes sa che desideriamo sempre tre repliche.

4. Controller Manager

A questo punto entra in gioco il Controller Manager.

Il suo compito consiste nel confrontare continuamente due situazioni.

Lo stato dichiarato.

E lo stato reale.

Supponiamo che il Deployment richieda.

```
3 Pod
```

Il Controller osserva il cluster.

Trova soltanto.

```
1 Pod
```


Individua quindi una differenza.

Questa differenza prende il nome di **drift**.

Il Controller decide quindi di intervenire.

Non crea direttamente il Pod.

Ordina semplicemente che venga creato.

Diagramma – Il reconciliation loop

Figura 2.9 – Il Controller confronta continuamente lo stato reale con quello desiderato.

5. Scheduler

Adesso il cluster sa che deve creare un nuovo Pod.

Ma dove?

In un cluster composto da più nodi questa decisione è tutt'altro che banale.

Lo Scheduler analizza numerosi fattori.

Tra cui:

- CPU disponibile;
- memoria disponibile;
- vincoli dichiarati nei manifest;
- affinità;
- anti-affinità;
- taint e toleration.

Nel nostro laboratorio utilizziamo un solo nodo.

La scelta è quindi immediata.

Nei cluster aziendali, invece, lo Scheduler rappresenta uno dei componenti più sofisticati dell'intera piattaforma.

6. kubelet

Ogni nodo del cluster esegue un processo chiamato **kubelet**.

Il kubelet riceve dallo Scheduler l'ordine di creare un nuovo Pod.

A questo punto verifica che tutto sia disponibile.

Se necessario scarica l'immagine Docker.

Crea il container.

Avvia il processo.

Infine comunica nuovamente lo stato al Control Plane.

7. Container Runtime

Il kubelet non crea direttamente i container.

Per questa attività utilizza un Container Runtime.

Nelle versioni moderne di Kubernetes il runtime più diffuso è **containerd**.

Dal punto di vista dello sviluppatore questa differenza è quasi trasparente.

È comunque utile sapere che Kubernetes non crea direttamente i container.

Delega questa attività a un runtime specializzato.

Finalmente il Pod

Solo adesso il Pod entra realmente in esecuzione.

Il Deployment passa allo stato disponibile.

Il Service aggiorna automaticamente i propri Endpoint.

L'applicazione torna nello stato desiderato.

L'intero processo richiede generalmente pochi secondi.

Un dettaglio molto importante

Probabilmente hai notato una cosa.

In nessun momento Kubernetes ha eseguito il Deployment.

Ha eseguito il Pod.

Il Deployment rappresenta soltanto una descrizione.

Il Pod rappresenta l'oggetto realmente eseguito sul nodo.

Questa distinzione sarà fondamentale quando introdurremo Helm e OpenShift.

Best Practice

Quando qualcosa non funziona chiediti sempre:

In quale punto del flusso si è verificato il problema?

Per esempio.

- Il manifest è stato rifiutato?
- L'API Server ha restituito un errore?
- Lo Scheduler non trova un nodo disponibile?
- Il kubelet non riesce ad avviare il container?
- L'immagine Docker non è raggiungibile?

Imparare a seguire questo percorso permette di diagnosticare rapidamente la maggior parte dei problemi.

Cosa vedremo nel prossimo paragrafo

Adesso conosciamo tutti gli elementi fondamentali di Kubernetes.

È arrivato il momento di metterli insieme.

Nel prossimo laboratorio distribuiremo l'intera applicazione BookStore utilizzando:

- Namespace;
- Deployment;
- Service;
- Pod.

Per la prima volta lavoreremo come farebbe un team di sviluppo in un progetto reale.

2.9 Laboratorio – Distribuiamo BookStore su Kubernetes

Nei paragrafi precedenti abbiamo studiato ogni componente separatamente.

Abbiamo imparato che cosa sono:

- i Pod;
- i Deployment;
- i Service;
- i Namespace.

È arrivato il momento di utilizzare tutti questi elementi insieme.

L'obiettivo di questo laboratorio è distribuire l'intera applicazione **BookStore** all'interno del cluster Kubernetes.

Al termine avremo un'infrastruttura molto simile a quella che utilizzeremo successivamente in OpenShift.

L'architettura finale

BookStore sarà composto da tre componenti principali.



Nel PDF finale questo schema verrà sostituito da un diagramma vettoriale.

È importante osservare che ogni componente possiede una responsabilità ben precisa.

Organizzazione del progetto

La struttura della directory Kubernetes inizia a diventare più articolata.

```
kubernetes/  
  
├─ namespace/  
  
|   └─ namespace.yaml  
  
├─ deployments/  
  
|   ├── backend-deployment.yaml  
  
|   ├── frontend-deployment.yaml  
  
|   └─ postgres-deployment.yaml  
  
├─ services/  
  
|   ├── backend-service.yaml  
  
|   ├── frontend-service.yaml  
  
|   └─ postgres-service.yaml
```

Questa organizzazione verrà mantenuta per tutto il libro.

Quando introdurremo Helm e OpenShift continueremo ad utilizzare la stessa struttura.

Ordine di distribuzione

Un errore molto comune consiste nell'applicare i manifest in ordine casuale.

Per il momento seguiremo sempre questa sequenza.

1. Namespace
2. Database
3. Backend
4. Frontend

Questa scelta permette ai componenti di trovare immediatamente le dipendenze necessarie.

Creazione del Namespace

Distribuiamo il Namespace.

```
kubectl apply \  
-f namespace/namespace.yaml
```

Verifichiamo.

```
kubectl get namespaces
```

Output.

```
bookstore
```

Distribuzione di PostgreSQL

Applichiamo il Deployment.

```
kubectl apply \  
-f deployments/postgres-deployment.yaml
```

Successivamente creiamo il Service.

```
kubectl apply \  
-f services/postgres-service.yaml
```

Controlliamo.

```
kubectl get pods \  
-n bookstore
```

Output.

```
postgres-5c8dfb9b74-82plx  
  
Running
```

Il database è pronto.

Distribuzione del backend

Applichiamo il Deployment.

```
kubectl apply \  
-f deployments/backend-deployment.yaml
```

Creiamo il Service.

```
kubectl apply \  
-f services/backend-service.yaml
```

Controlliamo.

```
kubectl get deployments \  
-n bookstore
```

Output.

```
bookstore-backend  
  
1/1
```

A questo punto il backend è operativo.

Può comunicare con PostgreSQL attraverso il Service.

Non utilizza alcun indirizzo IP.

La connessione avviene semplicemente tramite il nome.

```
postgres-service
```

Distribuzione del frontend

Ripetiamo la stessa procedura.

```
kubectl apply \  
-f deployments/frontend-deployment.yaml
```

```
kubectl apply \  
-f services/frontend-service.yaml
```

Verifichiamo.

```
kubectl get all \  
-n bookstore
```

Otterremo una panoramica completa.

```
Pods  
  
Deployments  
  
ReplicaSets  
  
Services
```

Per la prima volta il cluster contiene un'applicazione completa.

Analizziamo il risultato

Fermiamoci qualche minuto.

Osserva attentamente ciò che abbiamo costruito.

Il frontend non conosce il backend.

Conosce soltanto il Service.

Il backend non conosce PostgreSQL.

Conosce soltanto il Service.

Ogni componente comunica esclusivamente attraverso indirizzi stabili.

Questo rende l'intera architettura indipendente dal ciclo di vita dei Pod.

È proprio questo uno dei principi fondamentali di Kubernetes.

Controlliamo il cluster

Durante lo sviluppo abituiamoci a utilizzare sempre questa sequenza di controlli.

```
kubectl get deployments \  
-n bookstore
```

```
kubectl get pods \  
-n bookstore
```

```
kubectl get services \  
-n bookstore
```

```
kubectl get events \  
-n bookstore
```

Questi quattro comandi permettono di individuare rapidamente la maggior parte dei problemi.

Nei capitoli successivi diventeranno parte della nostra routine quotidiana.

Verifica del laboratorio

Prima di proseguire assicurati che il cluster soddisfi tutte le seguenti condizioni.

- Il Namespace `bookstore` esiste.
- PostgreSQL è nello stato `Running`.
- Il backend è nello stato `Running`.
- Il frontend è nello stato `Running`.
- Tutti i Service risultano creati correttamente.
- Nessun Pod è nello stato `CrashLoopBackOff`.

Se una di queste condizioni non è soddisfatta interrompi il laboratorio e individua il problema.

È fondamentale comprendere ogni passaggio prima di procedere.

Best Practice

Quando distribuisce un'applicazione non limitarti a verificare che i Pod siano nello stato **Running**.

Controlla sempre anche:

- Deployment;
- ReplicaSet;
- Service;
- Events.

Molti problemi non sono immediatamente visibili osservando soltanto i Pod.

Cosa vedremo adesso

L'applicazione è finalmente operativa.

Ma possiamo renderla ancora più robusta.

Nel prossimo laboratorio eseguiremo una serie di esperimenti.

Romperemo volutamente il sistema.

Elimineremo Pod.

Aumenteremo il numero delle repliche.

Aggiornaremo il backend.

Infine torneremo automaticamente alla versione precedente.

Per la prima volta vedremo Kubernetes comportarsi come un vero orchestratore.

2.10 Laboratorio – Lascia che Kubernetes faccia il suo lavoro

Nei paragrafi precedenti abbiamo costruito l'applicazione BookStore.

Abbiamo creato:

- un Namespace;
- tre Deployment;
- tre Service;
- i Pod necessari per eseguire frontend, backend e database.

L'applicazione è operativa.

Adesso faremo qualcosa di insolito.

La romperemo.

Più volte.

L'obiettivo non è distruggere il cluster.

L'obiettivo è osservare come Kubernetes reagisce automaticamente ai problemi.

Questo laboratorio rappresenta il primo vero contatto con il modello dichiarativo.

Obiettivo

Comprendere il funzionamento del **reconciliation loop** osservando il comportamento del cluster durante alcune situazioni di errore.

Tempo stimato

20 minuti

Prerequisiti

- Cluster Kubernetes funzionante.
 - Namespace `bookstore` .
 - Frontend, backend e PostgreSQL nello stato `Running` .
-

Esperimento 1 – Eliminiamo un Pod

Visualizziamo il backend.

```
kubectl get pods -n bookstore
```

Supponiamo di ottenere.

```
NAME
```

```
bookstore-backend-6d5cfb89df-1x8pw
```

```
postgres-7b9dcd76c7-g7m5k
```

```
frontend-79dd5f5c87-17fg8
```

Eliminiamo il Pod del backend.

```
kubectl delete pod bookstore-backend-6d5cfb89df-1x8pw \
-n bookstore
```

L'output sarà.

```
pod "bookstore-backend-6d5cfb89df-1x8pw" deleted
```

Adesso osserviamo continuamente il cluster.

```
kubectl get pods \
-n bookstore \
--watch
```

Dopo pochi secondi vedremo comparire un nuovo Pod.

bookstore-backend-6d5cfb89df-1x8pw

↓

Terminating

↓

bookstore-backend-7849fd56db-kd2xw

Running

Il Pod precedente è stato eliminato.

Il nuovo Pod possiede un nome differente.

Anche l'indirizzo IP sarà differente.

Eppure l'applicazione continua a funzionare.

Cosa è successo?

Il Deployment osserva continuamente il numero di Pod realmente disponibili.

Lo stato desiderato prevede una replica.

Lo stato reale, dopo l'eliminazione del Pod, è pari a zero.

Il Controller individua questa differenza.

Ordina quindi al ReplicaSet di creare una nuova replica.

L'intero processo richiede generalmente pochi secondi.

Nessuno sviluppatore è intervenuto.

Nessun amministratore di sistema ha eseguito ulteriori comandi.

È stato Kubernetes a riportare automaticamente il sistema nello stato dichiarato.

Esperimento 2 – Aumentiamo le repliche

Visualizziamo il Deployment.

```
kubectl get deployment \  
-n bookstore
```

Adesso chiediamo al cluster di mantenere tre copie del backend.

```
kubectl scale deployment bookstore-backend \  
--replicas=3 \  
-n bookstore
```

Controlliamo il risultato.

```
kubectl get pods \  
-n bookstore
```

Output.

```
bookstore-backend-a1234
```

```
bookstore-backend-b5678
```

```
bookstore-backend-c9012
```

Il backend è ora composto da tre Pod distinti.

Tutti eseguono la stessa applicazione.

Ma il frontend?

È una domanda molto interessante.

Il frontend continua a utilizzare lo stesso indirizzo.

```
http://bookstore-backend-service:8080
```

Non deve conoscere i tre Pod.

Non deve essere modificato.

Il Service distribuisce automaticamente il traffico verso una delle repliche disponibili.

Questa separazione delle responsabilità rappresenta uno dei maggiori vantaggi dell'architettura Kubernetes.

Esperimento 3 – Riduciamo le repliche

Torniamo ad una sola replica.

```
kubectl scale deployment bookstore-backend \
--replicas=1 \
-n bookstore
```

Visualizziamo nuovamente i Pod.

```
kubectl get pods \
-n bookstore
```

Due Pod verranno eliminati.

Uno soltanto rimarrà operativo.

Ancora una volta non abbiamo eliminato direttamente alcun Pod.

Abbiamo semplicemente modificato lo stato desiderato.

Kubernetes ha fatto tutto il resto.

Esperimento 4 – Aggiorniamo il backend

Immaginiamo di avere una nuova versione dell'applicazione.

Prima.

```
bookstore/backend:1.0
```

Ora.

```
bookstore/backend:1.1
```

Modifichiamo il Deployment.

```
image:
```

```
bookstore/backend:1.1
```

Applichiamo nuovamente il manifest.

```
kubectl apply \  
-f backend-deployment.yaml
```

Verifichiamo.

```
kubectl rollout status \  
deployment/bookstore-backend \  
-n bookstore
```

Output.

```
deployment "bookstore-backend"
```

```
successfully rolled out
```

Rolling Update

Kubernetes non elimina improvvisamente tutti i Pod.

L'aggiornamento avviene gradualmente.

Possiamo rappresentarlo così.

```
Backend v1.0
```

↓

```
Nuovo Pod v1.1
```

↓

Verifica

↓

Eliminazione vecchio Pod

↓

Nuovo Pod v1.1

↓

Verifica

↓

Eliminazione vecchio Pod

Questo meccanismo prende il nome di **Rolling Update**.

Permette di aggiornare l'applicazione mantenendo il servizio disponibile.

Esperimento 5 – Rollback

Immaginiamo che la nuova versione contenga un errore.

Possiamo tornare rapidamente alla versione precedente.

```
kubectl rollout undo \  
deployment/bookstore-backend \  
-n bookstore
```

Controlliamo lo storico.

```
kubectl rollout history \  
deployment/bookstore-backend \  
-n bookstore
```

Output.

```
REVISION
```

```
1
```

```
2
```

Kubernetes conserva automaticamente le revisioni del Deployment.

Questo rende il rollback estremamente semplice.

Come si fa in azienda

Negli ambienti di produzione gli sviluppatori raramente eseguono manualmente i comandi:

- `kubectl scale`
- `kubectl rollout`
- `kubectl rollout undo`

Queste operazioni vengono normalmente eseguite da pipeline CI/CD oppure da strumenti GitOps.

Per esempio.

1. Uno sviluppatore esegue un commit su Git.
2. La pipeline costruisce una nuova immagine Docker.
3. Argo CD rileva la modifica del manifest.
4. Kubernetes applica automaticamente il Rolling Update.

Dal punto di vista del cluster non cambia nulla.

Continua semplicemente a mantenere lo stato dichiarato nel repository Git.

Nei prossimi capitoli costruiremo esattamente questo flusso.

Lo sapevi?

Eliminare un Pod rappresenta uno dei test più semplici per verificare che un Deployment sia configurato correttamente.

Se il Pod viene ricreato automaticamente significa che il modello dichiarativo sta funzionando come previsto.

Cosa abbiamo imparato?

Con pochi semplici esperimenti abbiamo osservato alcune delle caratteristiche che rendono Kubernetes uno strumento così potente.

Abbiamo visto che il cluster è in grado di:

- ricreare automaticamente un Pod eliminato;
- aumentare e ridurre il numero delle repliche;
- aggiornare gradualmente un'applicazione;
- ripristinare rapidamente una versione precedente.

Queste funzionalità non richiedono script personalizzati.

Sono parte integrante del funzionamento di Kubernetes.

Nei capitoli dedicati a OpenShift e GitOps utilizzeremo continuamente questi meccanismi, senza dover intervenire manualmente sul cluster.

2.11 Errori comuni

Dopo aver distribuito alcune applicazioni su Kubernetes è normale imbattersi in problemi che, inizialmente, possono sembrare difficili da interpretare.

La buona notizia è che la maggior parte degli errori commessi dai principianti si ripete continuamente.

Conoscerli permette di risparmiare molto tempo durante le attività di sviluppo e di troubleshooting.

In questa sezione analizzeremo gli errori più frequenti e il modo corretto di affrontarli.

Errore 1 – Creare direttamente un Pod

Molti sviluppatori iniziano distribuendo esclusivamente Pod.

```
kind: Pod
```

Questo approccio è utile per comprendere il funzionamento di Kubernetes.

Non rappresenta però una buona pratica per applicazioni reali.

Se il Pod viene eliminato, Kubernetes non lo ricrea automaticamente.

La soluzione consiste nell'utilizzare un Deployment.

```
kind: Deployment
```

Il Deployment manterrà automaticamente il numero desiderato di repliche.

Errore 2 – Utilizzare l'indirizzo IP di un Pod

Dopo aver visualizzato il risultato del comando

```
kubectl get pods -o wide
```

molti sviluppatori configurano il frontend utilizzando direttamente l'indirizzo IP del backend.

Per esempio.

```
http://10.244.0.12:8080
```

Questa configurazione funziona soltanto fino a quando il Pod rimane in esecuzione.

Non appena il Pod viene sostituito, l'indirizzo IP cambia.

L'applicazione smette immediatamente di funzionare.

La soluzione consiste nell'utilizzare sempre il nome del Service.

```
http://bookstore-backend-service:8080
```

Il Service rappresenta il punto di accesso stabile all'applicazione.

Errore 3 – Dimenticare il Namespace

Supponiamo di distribuire il backend nel Namespace `bookstore`.

Successivamente eseguiamo.

```
kubectl get pods
```

Output.

```
No resources found.
```

Molti sviluppatori pensano immediatamente che il Deployment non abbia funzionato.

In realtà stanno semplicemente osservando il Namespace predefinito.

Il comando corretto è.

```
kubectl get pods -n bookstore
```

oppure, ancora meglio.

```
kubectl config set-context \  
--current \  
--namespace=bookstore
```

Errore 4 – Modificare manualmente i Pod

Immaginiamo di voler cambiare l'immagine Docker del backend.

Un errore molto comune consiste nell'eliminare il Pod e crearne uno nuovo manualmente.

Questo approccio è contrario alla filosofia di Kubernetes.

Le modifiche devono essere applicate al Deployment.

Sarà il Deployment ad aggiornare automaticamente i Pod.

Errore 5 – Ignorare gli Events

Quando un Pod non parte, molti sviluppatori iniziano immediatamente a modificare il manifest YAML.

Nella maggior parte dei casi è una cattiva idea.

La prima operazione dovrebbe sempre essere.

```
kubectl describe pod <nome-pod>
```

La sezione **Events** racconta tutto ciò che Kubernetes ha tentato di fare.

Moltissimi problemi possono essere individuati semplicemente leggendo questa parte dell'output.

Errore 6 – Non leggere i log

Uno stato **Running** non garantisce che l'applicazione stia funzionando correttamente.

Il processo principale potrebbe terminare immediatamente dopo l'avvio.

Per questo motivo è sempre consigliabile controllare anche i log.

```
kubectl logs <nome-pod>
```

Nel caso di applicazioni Spring Boot questa operazione permette di individuare rapidamente errori di configurazione, problemi di connessione al database o eccezioni durante l'avvio.

Errore 7 – Pensare che Kubernetes esegua i Deployment

Uno degli equivoci più frequenti riguarda il ruolo del Deployment.

Il Deployment non viene mai eseguito sul nodo.

Il Deployment descrive semplicemente lo stato desiderato.

Le risorse realmente eseguite sono i Pod.

Comprendere questa distinzione permette di interpretare correttamente il comportamento del cluster.

Errore 8 – Utilizzare kubectl come strumento di amministrazione

Durante i laboratori abbiamo utilizzato frequentemente il comando.

```
kubectl apply
```

Negli ambienti aziendali questo comando viene usato molto meno di quanto si possa immaginare.

Normalmente il flusso è il seguente.

Lo sviluppatore modifica un manifest.

Esegue il commit sul repository Git.

Una pipeline aggiorna automaticamente il cluster.

Con l'introduzione di Argo CD vedremo che sarà il repository Git a rappresentare la sorgente della verità.

Lo sviluppatore non dovrà più modificare direttamente il cluster.

Riepilogo

Quasi tutti gli errori appena descritti derivano dalla stessa causa.

Si continua a ragionare come se Kubernetes fosse Docker.

In realtà Kubernetes introduce un modello completamente differente.

Lo sviluppatore non gestisce più direttamente i container.

Descrive semplicemente lo stato desiderato dell'applicazione.

Il cluster farà il resto.

Da ricordare

Quando qualcosa non funziona evita di chiederti:

"Quale comando devo eseguire?"

Chiediti invece:

"Quale risorsa del cluster non corrisponde più allo stato dichiarato?"

Questo semplice cambio di prospettiva rappresenta il passaggio più importante per diventare realmente produttivi con Kubernetes.

2.12 Best Practice

Durante questo capitolo abbiamo costruito l'applicazione BookStore partendo da semplici Pod fino ad arrivare a un'architettura composta da Namespace, Deployment e Service.

Oltre ai concetti teorici, è importante acquisire alcune abitudini che renderanno il lavoro quotidiano molto più semplice.

Le seguenti Best Practice derivano dall'esperienza della community Kubernetes e dalla documentazione ufficiale del progetto.

1. Utilizza sempre manifest dichiarativi

Evita di creare risorse esclusivamente tramite comandi imperativi.

Per esempio.

```
kubectl run nginx
```

può essere utile durante un laboratorio.

Per un'applicazione reale è preferibile utilizzare un manifest YAML.

```
apiVersion: apps/v1
kind: Deployment
...
```

Il manifest può essere:

- versionato con Git;
- revisionato;
- condiviso con il team;
- riutilizzato in ambienti differenti.

Questo approccio rappresenta la base dell'Infrastructure as Code e sarà fondamentale quando introdurremo Helm e Argo CD.

2. Distribuisci sempre tramite Deployment

Un Pod dovrebbe essere creato direttamente soltanto durante attività di studio, debugging o sperimentazione.

Per tutte le applicazioni reali utilizza un Deployment.

Il Deployment offre automaticamente:

- riconciliazione;
 - gestione delle repliche;
 - rolling update;
 - rollback;
 - integrazione con gli strumenti GitOps.
-

3. Non utilizzare mai gli indirizzi IP dei Pod

Gli indirizzi IP dei Pod sono temporanei.

Possono cambiare ogni volta che un Pod viene ricreato.

Le applicazioni dovrebbero comunicare esclusivamente tramite i Service.

Corretto.

```
http://bookstore-backend-service:8080
```

Da evitare.

```
http://10.244.0.12:8080
```

4. Organizza il cluster mediante Namespace

Evita di distribuire applicazioni nel Namespace `default`.

Crea un Namespace dedicato per ogni applicazione oppure per ogni ambiente.

Per esempio.


```
bookstore-dev
```

```
bookstore-test
```

```
bookstore-prod
```

Questa organizzazione semplifica notevolmente la gestione del cluster e prepara il terreno all'utilizzo di OpenShift, dove i Namespace vengono rappresentati come **Project**.

5. Utilizza Labels in modo coerente

Le Labels rappresentano uno dei meccanismi più potenti di Kubernetes.

Mantieni una convenzione comune per tutto il progetto.

Per esempio.

```
labels:
```

```
  app: bookstore
```

```
  component: backend
```

```
  environment: development
```

Un utilizzo coerente delle Labels semplifica:

- il routing dei Service;
- il monitoraggio;
- le policy di sicurezza;
- il troubleshooting.

6. Controlla sempre lo stato del cluster

Prima di modificare un manifest verifica sempre lo stato corrente delle risorse.

I comandi più utilizzati dovrebbero diventare parte della tua routine.

```
kubectl get pods
```

```
kubectl get deployments
```

```
kubectl get services
```

```
kubectl get events
```

Quando una risorsa presenta un comportamento inatteso utilizza immediatamente.

```
kubectl describe
```

e

```
kubectl logs
```

Nella maggior parte dei casi contengono già tutte le informazioni necessarie per individuare il problema.

7. Versiona tutto

Non limitarti a versionare il codice dell'applicazione.

Versiona anche:

- Deployment;
- Service;
- Namespace;
- ConfigMap;
- Secret (utilizzando strumenti adeguati);
- documentazione.

L'intera infrastruttura dovrebbe poter essere ricreata partendo esclusivamente dal repository Git.

8. Pensa sempre in modo dichiarativo

Questo rappresenta probabilmente il consiglio più importante dell'intero capitolo.

Quando lavori con Kubernetes evita di chiederti.

"Quale comando devo eseguire?"

Chiediti invece.

"Quale stato desidero ottenere?"

Questa semplice differenza di approccio costituisce il fondamento di Kubernetes.

Nei capitoli dedicati a GitOps vedremo che anche Argo CD applica esattamente lo stesso principio.

Il repository Git descrive lo stato desiderato.

Il cluster lavora continuamente per raggiungerlo.

Conclusioni del capitolo

In questo capitolo hai compiuto il passaggio più importante dell'intero percorso.

Hai smesso di considerare Kubernetes come uno strumento che esegue comandi.

Hai iniziato a considerarlo come un sistema che mantiene automaticamente uno stato dichiarato.

Hai imparato a utilizzare:

- Namespace;
- Pod;
- Deployment;
- Service.

Hai osservato il funzionamento del reconciliation loop.

Hai distribuito l'applicazione BookStore all'interno di un cluster Kubernetes locale.

Nei capitoli successivi utilizzeremo queste conoscenze per introdurre OpenShift.

Scoprirai che OpenShift non sostituisce Kubernetes.

Lo estende con strumenti pensati per semplificare il lavoro quotidiano di sviluppatori e platform engineer.

Da questo momento tutte le tecnologie che incontreremo — Helm, Tekton e Argo CD — si baseranno sui concetti appresi in questo capitolo.

Il modo migliore per affrontare il resto del percorso è continuare a sperimentare.

Rompi il cluster.

Ricrealo.

Modifica i manifest.

Osserva il comportamento di Kubernetes.

È proprio attraverso questi esperimenti che si acquisisce la mentalità necessaria per lavorare efficacemente con piattaforme cloud-native.

Fonti ufficiali

Kubernetes

Documentazione ufficiale

<https://kubernetes.io/docs/>

Concetti fondamentali

<https://kubernetes.io/docs/concepts/>

Pod

<https://kubernetes.io/docs/concepts/workloads/pods/>

Deployment

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

ReplicaSet

<https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>

Service

<https://kubernetes.io/docs/concepts/services-networking/service/>

Namespace

<https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>

Labels e Selector

<https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/>

kubectl Cheat Sheet

<https://kubernetes.io/docs/reference/kubectl/cheatsheet/>

Kind

<https://kind.sigs.k8s.io/>

Minikube

<https://minikube.sigs.k8s.io/>

Docker

Docker Documentation

<https://docs.docker.com/>

Docker Compose

<https://docs.docker.com/compose/>

Docker Hub

<https://hub.docker.com/>

CNCF

Cloud Native Computing Foundation

<https://www.cncf.io/>

Cloud Native Landscape

<https://landscape.cncf.io/>

Documentazione OpenShift (anticipazione dei prossimi capitoli)

OpenShift Documentation

https://docs.redhat.com/en/documentation/openshift_container_platform

OpenShift Local (CRC)

<https://developers.redhat.com/products/openshift-local>

Helm

<https://helm.sh/docs/>

Tekton

<https://tekton.dev/docs/>

Argo CD

<https://argo-cd.readthedocs.io/>

GitOps

OpenGitOps

<https://opengitops.dev/>

Repository utilizzati durante il libro

Kubernetes GitHub

<https://github.com/kubernetes/kubernetes>

Kind GitHub

<https://github.com/kubernetes-sigs/kind>

Helm GitHub

<https://github.com/helm/helm>

Tekton GitHub

<https://github.com/tektoncd>

Argo CD GitHub

<https://github.com/argoproj/argo-cd>